

Phys Enph 479/879 Final Project: Reinforced Learning using Deep Q-Network to swing up and balance the inverted flywheel pendulum

Brendan Halliday^{1,*}

¹*Department of Physics, Engineering Physics and Astronomy,
Queen's University, Kingston, ON K7L 3N6, Canada*

(Dated: April 24, 2024)

In this paper, a Reinforced Learning (RL) Deep Q Network (DQN) algorithm was implemented to swing up and balance the inverted flywheel pendulum. A physics model was derived and implemented into a subclass of OpenAI's Gymnasium Environment class, which was used as the training environment for the RL agent. The Pytorch library was used for training the neural network on a single CPU. Animations of the fully trained model were produced and the entire code has been packaged and is now available open source.

I. INTRODUCTION

A. Background and Motivation

Reinforced learning is a subfield of machine learning where an agent is trained to complete a task by maximizing a numerical reward. Abstractly, the agent is given a set of actions and an environment in which to perform said actions. The environment is defined as the space of all possible states that the agent can occupy. Actions are taken to translate the agent from one state to another. By performing actions in the environment, the agent accumulates reward. Reward is a real number that can take on both positive and negative values. If the agent performs an action, translating it to favourable state, then a large reward is accumulated. Conversely, if the agent performs an action rendering it in an unfavourable state, a small, null or negative reward is accumulated. Reward is a function that maps the agents occupied states to a number. The learning agent is not given set instructions to complete the task, but rather it is tasked to discover the correct actions based on accumulating the maximal reward over a finite time interval [1].

Reinforced learning is often applied to trial-error situations to find the optimal actions for achieving a specified goal. An excellent example of reinforced learning is the AlphaZero model, a chess engine trained by playing 44 million games of chess by itself [2]. The only constraints placed on the system were the rules of chess, and remarkably, AlphaZero was able to surpass the StockFish algorithm, the previously reigning computer chess engine. In recent years, reinforced learning has been used in the field of control theory to manipulate and control complex physical systems such as the inverted pendulum.

B. Literature Review

Applying reinforced learning to balancing the inverted pendulum is not necessarily novel. This application has been implemented countless times for the 'cart-pole' model where a rigid rod is fixed to a cart that can translate in one

dimension. Qualitatively, the balancing act comes from controlling the acceleration of the cart to keep the rod balanced in its unstable equilibrium. This technique has even been applied to the double [3] and even the triple [4] inverted pendulum situation all using reinforced learning.

A similar technique will be implemented here, following the technique outlined by Israilov *et al.*[5] with the key difference being modelling the inverted pendulum without the cart but with 1 and 2 reaction fly wheels as the mechanism for dynamic balancing. In Israilov *et al.*[5], a Deep Q-Network model is proposed for training our learning agent.

A Deep Q-Network is a neural network that is based on the Q-learning algorithm to train a reinforced learning model. The Q learning algorithm uses the Bellman's equation to associate with each state a quality factor. This quality factor is a measure of how much cumulative reward will be gained given that the agent performs the best possible sequence of actions. The problem with this is that the quality factor must be calculated for all possible states given the possible actions allowed. This is computationally expensive as the number of states can be very large, especially if the state space is over a continuous interval. So it is the job of a neural network to approximate the quality function for all possible states[1, 6, 7].

The control of the inverted flywheel pendulum has been done experimentally using control methods such as Proportional Integral-Derivative (PID) [8, 9]. However, in both these cases, reinforced learning was not applied.

Perhaps the most similar model of this type applied to reinforced learning was found in the Gymnasium library for Python written by OpenAI. In this situation, the model directly drives a pendulum with some arbitrary torque. OpenAI has yet to publish the inverted flywheel pendulum as a class in its Gymnasium library.

The general physics for the balancing mechanism is described here: consider a rod of uniform linear mass density λ initially positioned vertically on a flat table. The rod is placed in an unstable equilibrium where some small perturbation would off balance the rod. Torque is applied by the force of gravity when the rod is pitched by some angle from the vertical axis. This torque is what turns the rod about its pivot point with the table. Torque can be applied in the opposite direction of that applied by gravity using a set of driven reaction fly wheels to cancel out the torque from gravity. This is dynamic balancing. For a rod that can only pivot in 1D, only one reaction wheel is necessary

* 17bsh3@queensu.ca

to cancel out τ_g . For a rod that can pivot in 2D, 2 reaction wheels are required. The target value for the reinforced learning algorithm will be keeping the rod upright by minimizing pitch from the central axis. Minimizing pitch will be associated with larger reward. The input parameters will be the acceleration of the top fly wheels in either the clockwise or counter clockwise directions.

In this paper, we present a novel application of reinforced learning by using a Deep Q-Network to swing up and balance the inverted flywheel pendulum. Since this is a physics based model, our agent will be placed under physical constraints, making this a physics informed machine learning model[10]. This project is suitable for the upper undergraduate level as it requires a reasonable amount of computational physics to model the pendulum and its movement given the physical constraints.

II. METHODS, THEORY AND EQUATIONS

A. Equations of Motion

In this section, we provide a derivation of the equations of motion for the inverted flywheel pendulum in 1 dimension. Consider a rigid rod of uniform mass density and total mass m_1 and length L as shown in 1. One end is fixed to the origin such that the rod can pivot freely. Consider also a disk of mass m_2 and radius R whose center is fixed to the free end of the rod but can rotate about its center.

The disk is driven by a motor, whose mass we will neglect relative to the mass of the rod and flywheel. The Euler-Lagrange equations for a driven system are [8]:

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \sum_{k=1}^n F_{i,k}, \quad (1)$$

where $\mathcal{L} = T - U$ is the Lagrangian defined as the difference between kinetic energy T and potential energy U . Generalized coordinates q_i , and generalized velocities $\dot{q}_i$ are defined using the number of system degrees of freedom. For our application, θ_1 and θ_2 are the generalized coordinates for the angular position of the rod and flywheel respectively. Finally, $\sum_{k=1}^n F_{i,k}$ is the net sum of generalized forces applied in the direction of q_i . For our application, the sum reduces to the applied torque τ from the motor on θ_2 . We can rewrite 1 as the following system of equations[9]:

$$\begin{aligned} \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\theta}_1} - \frac{\partial \mathcal{L}}{\partial \theta_1} &= 0, \\ \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\theta}_2} - \frac{\partial \mathcal{L}}{\partial \theta_2} &= \tau. \end{aligned}$$

The kinetic energy can be decomposed into the sum of the kinetic energy of the rod T_L , and the kinetic energy of the flywheel, T_R . Each kinetic energy can be further decomposed into $T_i = T_{i,t} + T_{i,r}$ where the subscripts t and r refer to translational and rotational energies, respectively.

For the rod and flywheel, the translational kinetic energies is given by:

$$T_{1,t} = \frac{1}{2} m_1 (\dot{\theta}_1 l_1)^2, T_{2,t} = \frac{1}{2} m_2 (\dot{\theta}_1 l_2)^2, \quad (2)$$

where $l_1 = L/2$, $l_2 = L$ and $\dot{\theta}_1$ is the angular velocity of the rod. The rotational kinetic energies are given by:

$$T_{1,r} = \frac{1}{2} I_1 (\dot{\theta}_1)^2, T_{2,r} = \frac{1}{2} I_2 (\dot{\theta}_1 + \dot{\theta}_2)^2, \quad (3)$$

where I_1 and I_2 are the moments of inertia of the rod and flywheel respectively and $\dot{\theta}_2$ is the angular velocity of the flywheel. Thus, we can write the combined kinetic energy of the system as:

$$T = \frac{1}{2} m_1 (\dot{\theta}_1 l_1)^2 + \frac{1}{2} I_1 (\dot{\theta}_1)^2 + \frac{1}{2} m_2 (\dot{\theta}_1 l_2)^2 + \frac{1}{2} I_2 (\dot{\theta}_1 + \dot{\theta}_2)^2. \quad (4)$$

Potential energy U is defined for both the rod and the flywheel and is given by:

$$U = -m_1 g l_1 \cos \theta_1 - m_2 g l_2 \cos \theta_2, \quad (5)$$

where $g \approx 9.81 \text{ m/s}^2$ is the acceleration due to gravity. Finally, the generalized forces for our driven system is given by the applied torque τ given to the flywheel. We can then write the Euler-Lagrange equations and isolate for $\ddot{\theta}_1$ and $\ddot{\theta}_2$ as:

$$\begin{aligned} \ddot{\theta}_1 &= \frac{-g \sin(\theta_1)(l_1 m_1 + l_2 m_2) - \tau}{I_1 + l_1^2 m_1 + l_2^2 m_2}, \\ \ddot{\theta}_2 &= \frac{g \sin(\theta_1)(l_1 m_1 + l_2 m_2) + \tau(I_1 + I_2 + l_1^2 m_1 + l_2^2 m_2)}{I_2 I_1 + I_2 I_2 l_1^2 m_1 + I_2 l_2^2 m_2}. \end{aligned}$$

For implementation, a python subclass of OpenAI's Environment class from the Gymnasium library containing update equations of the flywheel was written for this report. The environment can also be rendered, taking user input controls using the Pygame library. While this is not necessary for training, it provides the user with a fun Graphical User Interface (GUI) to compete with a trained agent.

The leapfrog method for integrating the equations of motion was used since it was noticed that the model exhibited nonphysical behaviour when using the Forward-Euler scheme. Nonphysical behaviour includes divergence of angular velocities at long timescales.

B. Q-Learning

In this section, we will succinct the Q-learning algorithm. Let S be the space of all possible states in our learning environment E , and let A , be the space of all possible actions. Let s_t be the state observed at time t . The learning agent executes an action a_t , transitioning the agent to state s_{t+1} , and accumulates a reward r_t . In the context of the flywheel, a_t can be any of the actions in A for all states in S . There exists a policy, $p(a_t | s_t)$, which is the probability of

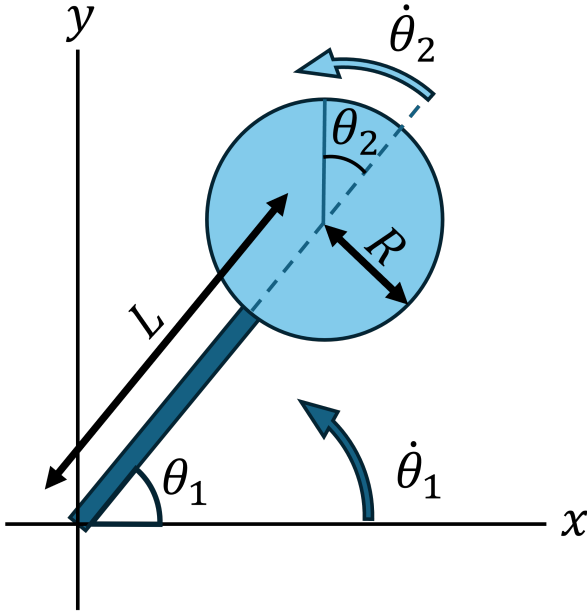


Figure 1. Geometry of inverted flywheel pendulum. Here L is the length of the rod, R is the flywheel radius, θ_1 and $\dot{\theta}_1$ are the angle and angular velocity of the rod and θ_2 and $\dot{\theta}_2$ are the angle and angular velocity of the driven flywheel

taking action a_t given state s_t . The objective of any reinforced learning task is to find the optimal policy, p^* , that if used to choose actions, will maximize the total cumulative reward. We can then define a discounted return function R_t that calculates the discounted cumulative reward gathered by the agent for all times after and including t such that [1, 5, 11]:

$$R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}, \quad (6)$$

where γ is the discount factor and T is the terminal time. The discount factor, $0 < \gamma < 1$, is a weighting such that rewards accumulated closer to time t have a larger effect on R_t . Since each r_t is dependent on its associated s_t and a_t pairs, we can next define a function $Q(s_t, a_t)$ as [5]:

$$Q(s_t, a_t) = \mathbb{E}(R_t | (s_t, a_t)), \quad (7)$$

which is the expectation value of R_t given a state-action pair (s_t, a_t) . Clearly, the optimal policy, is the one that selects the action $a^* = \arg \max_a (Q(s_i, a))$ that maximizes the expected cumulative reward given state s_i .

Since we are only given the starting state-action (s_t, a_t) and future state-action pairs are unknown, R_t is a random variable and we can compute its expectation value [12]. To accurately approximate the expectation value of the total return, we must sample the state-action space over a large number of state-action pairs. This can be computationally expensive as it requires many simulations, visiting many state-action pairs and associated rewards. Thus, as an approximation to 6, we write:

$$Q(s_t, a_t) \approx r_t + \gamma Q(s_{t+1}, a_{t+1}). \quad (8)$$

To solve for the Q function, we apply an iterative approach given by:

$$\frac{Q_{t+1} - Q_t}{\eta} = \Delta Q, \quad (9)$$

where we have dropped the s_t and a_t and instead written $Q_t = Q(s_t, a_t)$. η is actually the learning rate, but can act as the time step as in the Forward-Euler scheme [5]. Imposing that ΔQ satisfies:

$$\Delta Q = Q^* - Q_t, \quad (10)$$

ΔQ is strictly positive and our iterative equation 9 will always transform Q_t in the direction of optimal Q^* . The hypothesis made for Q-learning algorithm is that Q^* can be approximated as [1, 5]:

$$Q^* \approx r_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}), \quad (11)$$

where a_{t+1} is any action accessible to agent in state s_{t+1} and we can rewrite the iterative equation 9 as:

$$\begin{aligned} Q_{t+1} &= Q_t + \eta \Delta Q \\ \implies Q_{t+1} &= Q_t + \eta (r_t + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q_t). \end{aligned}$$

The dimension of the state-action space would be $N_s \times N_a$ where N_s is the number of possible states and N_a is the number of possible actions. Typically, the state space is very large and calculating an approximate Q^* can be computationally challenging since a large number of state-action pairs must be visited. Thus, in this paper, a neural network will be used to approximate the optimal Q^* . The DQN algorithm is similar to the Q learning algorithm, though there are some differences.

C. Deep-Q Networks

In the field of machine learning, Deep Neural Networks (DNN) or simply Neural Networks (NN) falls under a class of functions known as Universal Function Approximators (UFA) [13]. From approximating functions on continuous intervals [14] to recognizing and classifying handwritten digits [15–17], neural networks present a powerful tool for regression, pattern recognition and beyond.

The basic layout of a neural network is a series of interconnected nodes, organized in layers that transforms an input $\vec{x} \in \mathbb{R}^n$ to an output $\vec{y} \in \mathbb{R}^m$. Suppose we have a single layer, fully connected NN meaning each input node is connected to all nodes in the output. Our NN transforms $\vec{x}$ to $\vec{y}$ given by the following expression:

$$\vec{y} = f(\mathbf{W}\vec{x} + \vec{b}), \quad (12)$$

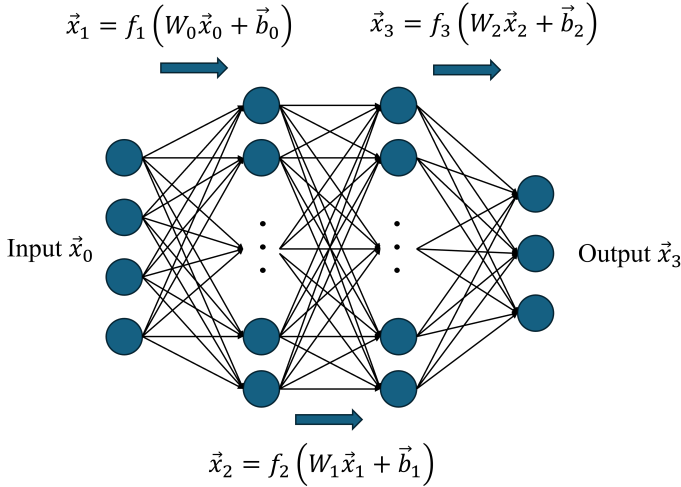


Figure 2. Diagram representing the fully connected neural network used in this report.

where $\mathbf{W} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a linear transformation whose matrix entries are weights, $\vec{b} \in \mathbb{R}^m$ is a bias vector, and $\vec{f} : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is the activation function defined as:

$$\vec{f} \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} \right) = \begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_n) \end{bmatrix}. \quad (13)$$

In 12 f is a one dimensional activation function which can take many forms. Typically, f can be an identity, a sigmoid (σ), a rectified linear unit (ReLU), or a hyperbolic tangent function.

The DNN used to approximate the Q-function for this application is shown in 2 as a 4 layer fully connected NN. The first layer of 2 shows a four node input $\vec{x}_0$ representing the 4 dimensional state vector. This is followed by two fully connected 128 node hidden layers $\vec{x}_1$ and $\vec{x}_2$. Finally, the output of the NN is given by a three node $\vec{x}_3$ representing the 3 dimensional action vector. The entries of $\vec{x}_3$ are the Q-values for each possible action: torque left, torque right or do nothing. Activation functions f_1 and f_2 are ReLU and f_3 is the identity function.

Typically, a loss function is computed to compare the NN's output prediction to known output labels. A gradient descent algorithm is then applied to the weights and biases of the NN to minimize the loss function. This is similar to least squares minimization used for curve fitting and regression analysis.

The choice of activation functions, number of layers and number of nodes per layer of the NN was based on the reinforced learning algorithm presented by Paszke and Towers [18] and applied to inverted flywheel pendulum. Their algorithm was used since it was capable of training the inverted pendulum on the cart-pole system. The 1 dimensional cart-pole system has essentially the same state-action space dimension as the inverted flywheel pendulum, hence their algorithm should be successful in this novel implementation. We will now explain how their algorithm works from a high level here.

Firstly, we initialize the reinforced learning environment defined in II A. We then define a Transition class who contains attributes such as the state, the action, the next state and the associated reward after each time step. A cyclic memory buffer is made to store the Transition objects so they can be sampled in batches for future DNN optimization. We then define the maximum number of training episodes to train the neural network.

Two NN are used in this algorithm: a policy NN and a target NN. The purpose of the policy NN is to represent the current Q-function Q_t from 7 that the agent will use to select actions based on a given state. The target NN is used to approximate the Q-function Q_{t+1} at a later time.

Now we begin the main training loop. For each new episode, we first reset the initial conditions of the learning environment. Now we can iterate until the simulation has terminated.

First we select an action based on the `selectaction(state)` function given the current state. Then, a step is taken given the selected action, translating the agent to a new state. A reward is accumulated and this transition (a state, action, next state, and reward vector) is stored in the memory buffer. The parameters of the policy NN are then updated using a `optimizemodel()` function. The parameters of the target network are then updated using the updated parameters of the policy NN using a soft update. This soft update is given by 14:

$$w_{target} = \tau w_{policy} + (1 - \tau) w_{target}, \quad (14)$$

where w_{target} and w_{policy} are the weights and biases of the target and policy networks, respectively, and $\tau = 0.005$ was chosen to be small. The soft update slightly updates the target nets weights by a small amount in the direction of the policy net. This is done since a soft update improves reinforced learning stability and performance [19].

The `selectaction(state)` function is defined in the following way. The function has access to both the current state and N the total number of steps done. The number of steps done is just the number of times the `selectaction(state)` function has been called after the user has started to run the program. The function first chooses a random number from the uniform distribution from 0 to 1 and store the value. A threshold number, ϵ_{thresh} , is defined as:

$$\epsilon_{thresh} = \epsilon_{End} + (\epsilon_{Start} - \epsilon_{End}) e^{-N/\epsilon_{Decay}}, \quad (15)$$

where $\epsilon_{End} = 0.05$, $\epsilon_{Start} = 0.95$ and $\epsilon_{Decay} = 1000$. If the previously chosen random number is greater than ϵ_{thresh} , then the agent will choose an action with the maximum Q value given by the policy network. If not, then the agent will choose an action at random from the action space. When N is small, the probability that a random action is taken is increased. The effect is that the agent is more likely to explore different combinations of states and actions, thus allowing it to evenly explore the state-action space at early times in the simulation. At later times, the agent will be less likely to explore, and will perform actions that are predicted to give the greatest cumulative reward by the policy NN.

The `optimizemodel(state)` function is defined in the following way. First, the function samples the memory buffer with a batch size of 128 transitions. From the 128 transition batch, each state (s) is transformed into an action vector using the policy NN. The action vector is a vector whose values are the Q values and whose indices are the associated actions. From the batch of action vectors the Q value of the action taken, (as stored in the transition object) is extracted resulting in a 128 length array of Q values. This array is the Q_{policy} array. Next the target NN is used to compute the action vectors of the batch of next states (s_{t+1}). From this action vector array, only the maximum component of each action vector is extracted resulting in another 128 length array. We then take the Q_{t+1} values from the target NN and compute the expectation value:

$$Q_{expect} = \gamma \max(Q_{t+1}) + r_t, \quad (16)$$

which is an approximation to Q^* from 11. The loss function is computed, which in this case is the Huber loss function defined as:

$$L = \begin{cases} \frac{1}{2}(Q_{expect} - Q_{policy})^2, & |Q_{expect} - Q_{policy}| < 1 \\ |Q_{expect} - Q_{policy}| - \frac{1}{2}, & \text{otherwise} \end{cases}$$

The gradients of the L with respect to the weights and biases of the policy NN are computed and then a single optimization step is computed to update the weights and biases using the AdamW optimizer. After each iteration of the main training loop, the policy NN makes increasingly better approximations to the optimal Q^* .

III. RESULTS

A. Balancing over-actuated inverted pendulum

The first training exercise was to balance the flywheel pendulum from an upright position. The initial position was set to be 0.1 rad from the positive y -axis. This test is most similar to the cart-pole system, hence, would make a good comparison to the two environments. After training the agent for 100 episodes, the agent could successfully balance the flywheel pendulum for the maximum simulation time of 30 seconds. The training process represented by duration vs. episode is shown in 3. The reward function was simply defined as the length of time that the agent can keep the pendulum above the horizontal axis; each time step a reward of 1 is returned. Physical parameters of the balance from upright are given in I under the fully actuated case. The maximum possible energy in this case was approximately 1775 J and the energy plot from 4 shows it is bounded by this maximum.

B. Swing up and balance underactuated inverted pendulum

Since successful balancing of the inverted pendulum could be achieved, the next step was to see if an agent could learn to swing up and then balance the inverted flywheel pendulum. From repeated trials, it was shown that

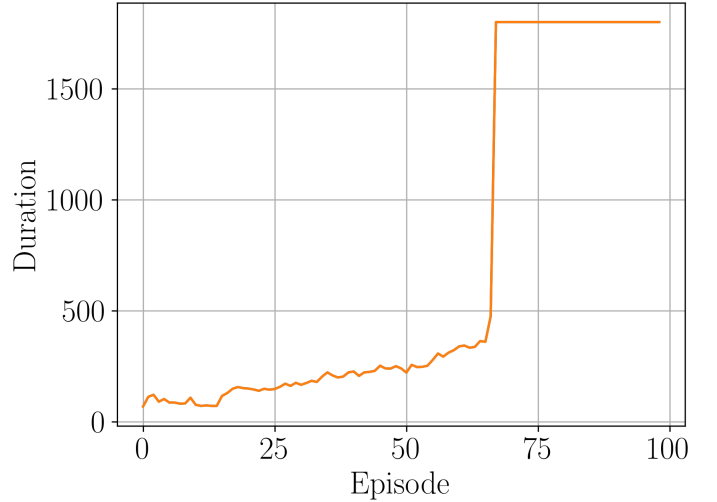


Figure 3. Duration of balance as a function of training episode for the balance from upright inverted pendulum. 1 unit of duration is 1/60 seconds, corresponding to the frame rate of the animation.

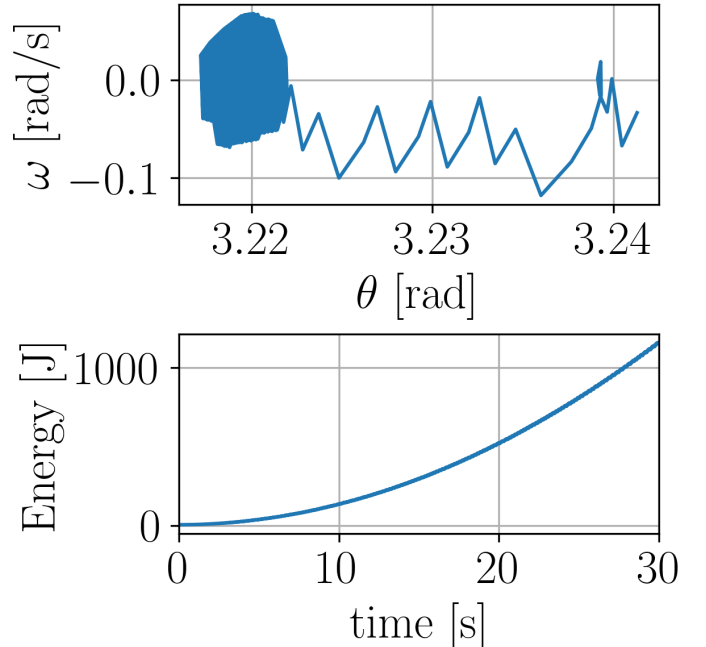


Figure 4. Phase diagram (top) and energy plot (bottom) of the balance from upright case. The angle θ and associated velocity ω describe angular position and velocity of the rod. In this phase plot (top) the pendulum starts at $\theta \approx 3.24$ rad and then shifts towards $\theta \approx 3.22$ rad where the phase trajectory becomes dense.

the learning agent could consistently swing up and balance the inverted pendulum within 100 training episodes. The cumulative reward was plotted as a function of episode number and averaged over 3 trials and the result is shown in 5. Physical properties of the flywheel system were chosen based on successful implementations of the inverted flywheel pendulum by Ashwin Krishna and Nao Ouyang [8] and are summarized in table I. In this case, the physical

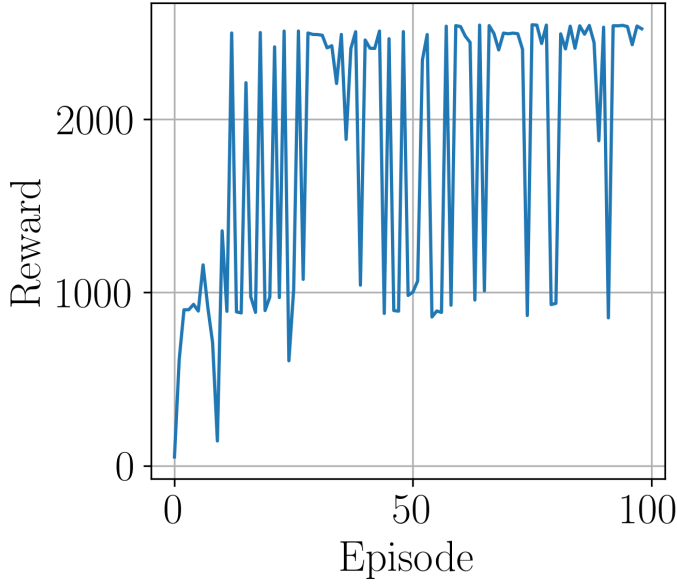


Figure 5. Cumulative reward as a function of training episode for the swing up and balance agent.

characteristics and max torque render the flywheel underactuated [20]. This means that if the agent were to apply a constant torque in one direction, the pendulum is not able to overcome gravity and transition to the upright position. The idea in the underactuated case: the agent must learn to swing the pendulum back and forth until it has enough energy to reach the upright vertical position.

In this case, the reward function was defined as:

$$r = \begin{cases} \frac{1}{2}(1 - \cos(\theta_1)) + \frac{1}{2}(1 - \frac{|\omega_1|}{\omega_{max}}), & \cos(\theta_1) < \cos(\pi + \frac{12\pi}{180}) \\ \frac{1}{2}(1 - \cos(\theta_1)), & \text{otherwise} \end{cases},$$

which rewards the agent more when the rod is closer to the positive y -axis and with a small speed. The second term in the first condition of III B is meant to prevent the agent from swinging the pendulum around its axis quickly, accumulating reward by doing so.

Physical Parameter	Underactuated	Fullyactuated
R [m]	0.085	0.3
l_1 [m]	0.105	0.35
l_2 [m]	0.21	0.7
m_1 [kg]	0.115	0.4
m_2 [kg]	0.526	0.4
$\max(\omega_1)$ [rad/s]	18.8	18.8
$\max(\omega_2)$ [rad/s]	418.9	418.9
$\max(\tau)$ [Nm]	0.215	0.8

Table I. Physical parameters of the underactuated and the fullyactuated inverted flywheel pendulums used in this study.

To show that the inverted flywheel system is in fact physical, plots of total energy and phase were plotted of a fully trained simulation and are shown in 6. From 6, the phase plot shows how the pendulum starts at $(0, 0)$ and then swings back and forth until it reaches the upright position

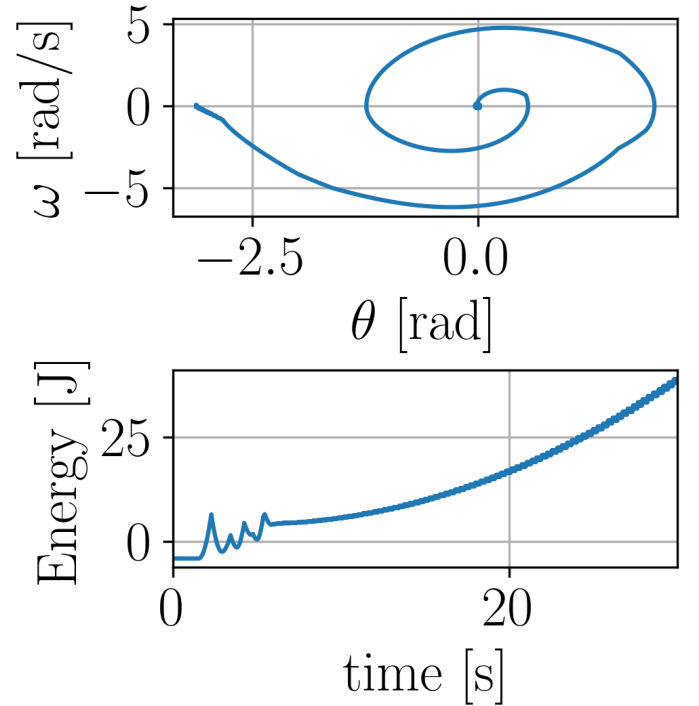


Figure 6. Phase diagram (top) and energy plot (bottom) of the balance from upright case. The angle θ and associated velocity ω describe angular position and velocity of the rod.

at $\theta = -\pi$ and zero angular velocity. The simulation lasted 30 seconds in real time, and the energy is bounded by the maximum possible energy of the underactuated flywheel system which is approximately 187 J.

IV. CONCLUSION AND FUTURE WORK

A DQN algorithm was used to successfully swing up and balance the inverted flywheel pendulum on a single CPU. Using Message Passing Interface (MPI) on a CPU to speed up matrix multiplication in the DNN training is inefficient compared to GPU training [21, 22]. The size of the model proved small enough that a single CPU could train the model in a relatively short time. However, a possible direction for future work would be to compare training performance on CPU vs GPU. The Double Flywheel Pendulum (DFP) would be another excellent next step for this project. The DFP would provide an excellent nonlinear system for a reinforced learning application. It is anticipated that such a system would take longer to train due to the increased state-action space size and, perhaps, would benefit from GPU parallelization.

V. ACKNOWLEDGEMENTS

The author would like to thank Rohan Jain and Muhammad Athar Ganaie, machine learning researchers, from the University of Calgary for providing guidance and useful discussions. All code has been made open source on GitHub.

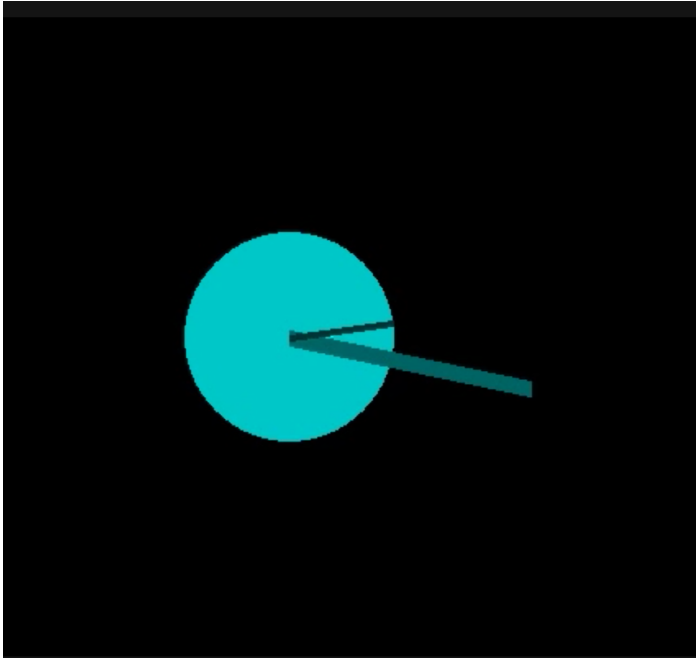


Figure 7. Animation screen capture of the fully trained DQN using PyGame

References

- [1] Richard S. Sutton and Andrew G. Barto, *Reinforced Learning: An Introduction* (The MIT Press, Cambridge, 2018).
- [2] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis, “Mastering chess and shogi by self-play with a general reinforcement learning algorithm,” (2017), [arXiv:1712.01815 \[cs.AI\]](#).
- [3] Michael Hesse, Julia Timmermann, Eyke Hüllermeier, and Ansgar Trächtler, “A reinforcement learning strategy for the swing-up of the double pendulum on a cart,” *Procedia Manufacturing* **24**, 15–20 (2018), 4th International Conference on System-Integrated Intelligence: Intelligent, Flexible and Connected Systems in Products and Production.
- [4] Jongchan Baek, Changhyeon Lee, Young Sam Lee, Soo Jeon, and Soohye Han, “Reinforcement learning to achieve real-time control of triple inverted pendulum,” *Engineering Applications of Artificial Intelligence* **128**, 107518 (2024).
- [5] Sador Israilov, Li Fu, Jesús Sánchez-Rodríguez, Franco Fusco, Guillaume Allibert, Christophe Raufaste, and Médéric Argentina, “Reinforcement learning approach to control an inverted pendulum: A general framework for educational purposes,” *PLOS ONE* **18**, 1–15 (2023).
- [6] Beakcheol Jang, Myeonghwi Kim, Gaspard Harerimana, and Jong Wook Kim, “Q-learning algorithms: A comprehensive classification and applications,” *IEEE Access* **7**, 133653–133667 (2019).
- [7] Christopher J. Watkins and Peter Dayan, “Q-learning,” *Machine Learning* **8**, 279–292 (1992).
- [8] Ashwin Krishna and Nao Ouyang, “Inertial wheel inverted pendulum,” (2019).
- [9] Petter Brevik, *Two-Axis Reaction Wheel Inverted Pendulum*, Masters thesis, Norwegian University of Science and Technology, Trondheim, Torgarden, Norway (2017), available at https://ntnuopen.ntnu.no/ntnu-xmlui/bitstream/handle/11250/2451060/12762_FULLTEXT.pdf?sequence=1.
- [10] Chayan Banerjee, Kien Nguyen, Clinton Fookes, and Maziar Raissi, “A survey on physics informed reinforcement learning: Review and open problems,” (2023), [arXiv:2309.01909 \[cs.LG\]](#).
- [11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, “Playing atari with deep reinforcement learning,” (2013), [arXiv:1312.5602 \[cs.LG\]](#).
- [12] Dimitri P. Bertsekas and John N. Tsitsiklis, “Introduction to probability,” (2008).
- [13] Kurt Hornik, Maxwell Stinchcombe, and Halbert White, “Multilayer feedforward networks are universal approximators,” *Neural Networks* **2**, 359–366 (1989).
- [14] K. Hornik, “Some new results on neural network approximation,” *Neural Networks* **6**, 1069–1072 (1993).
- [15] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE* **86**, 2278–2324 (1998).
- [16] Ziheng Wang, Su Wu, Chang Liu, Shaozhi Wu, and Kai Xiao, “The regression of mnist dataset based on convolutional neural network,” in *The International Conference on Advanced Machine Learning Technologies and Applications (AMLTA2019)*, edited by Aboul Ella Hassanien, Ahmad Taher Azar, Tarek Gaber, Roheet Bhatnagar, and Mohamed F. Tolba (Springer International Publishing, Cham, 2020) pp. 59–68.
- [17] Ruzhang Yang and Ruzhang Yang, “Classifying hand written digits with deep learning,” *Intelligent Information Management* **10**, 69–78 (2018).
- [18] Adam Paszke and Mark Towers, “Reinforcement learning (dqn) tutorial,” .
- [19] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra, “Continuous control with deep reinforcement learning,” (2019), [arXiv:1509.02971 \[cs.LG\]](#).
- [20] Russ Tedrake, *Underactuated Robotics* (2023).
- [21] Fidan Boylu Uz, “Gpus vs cpus for deployment of deep learning models — microsoft azure blog,” Talks about the differences between using CPUs and GPUs for training neural networks.
- [22] “Cuda c++ programming guide,” .